Chess Blunder-Risk Analyzer

Team Members: Braden Hayes, Nate Poppe

1. Title, Goal Statement, and Conventions

Chess Blunder-Risk Analyzer

Goal Statement

The goal of the Chess Blunder-Risk Analyzer is to develop a machine-learning-based system that estimates the likelihood that a chess player will make a blunder from a given position. The system will analyze completed chess games using factors such as player rating, clock time, game phase, position characteristics, and engine-derived information. It will then present move-level risk scores and broader patterns that help players identify situations in which they are more likely to make mistakes.

Basic Input and Output

The primary input to the system is a set of completed chess games, either imported from a user's Lichess account or provided through a PGN file. Each game includes the move sequence and available metadata such as player ratings, time control, and clock information.

The primary output is a move-by-move blunder-risk analysis. For each analyzed game, the system will identify the highest-risk moves and allow the user to inspect the position, predicted risk score, relevant risk factors, and Stockfish's preferred move. Across multiple games, the system may also identify recurring player-specific risk patterns.

2. Block Diagram — D0 High-Level Design

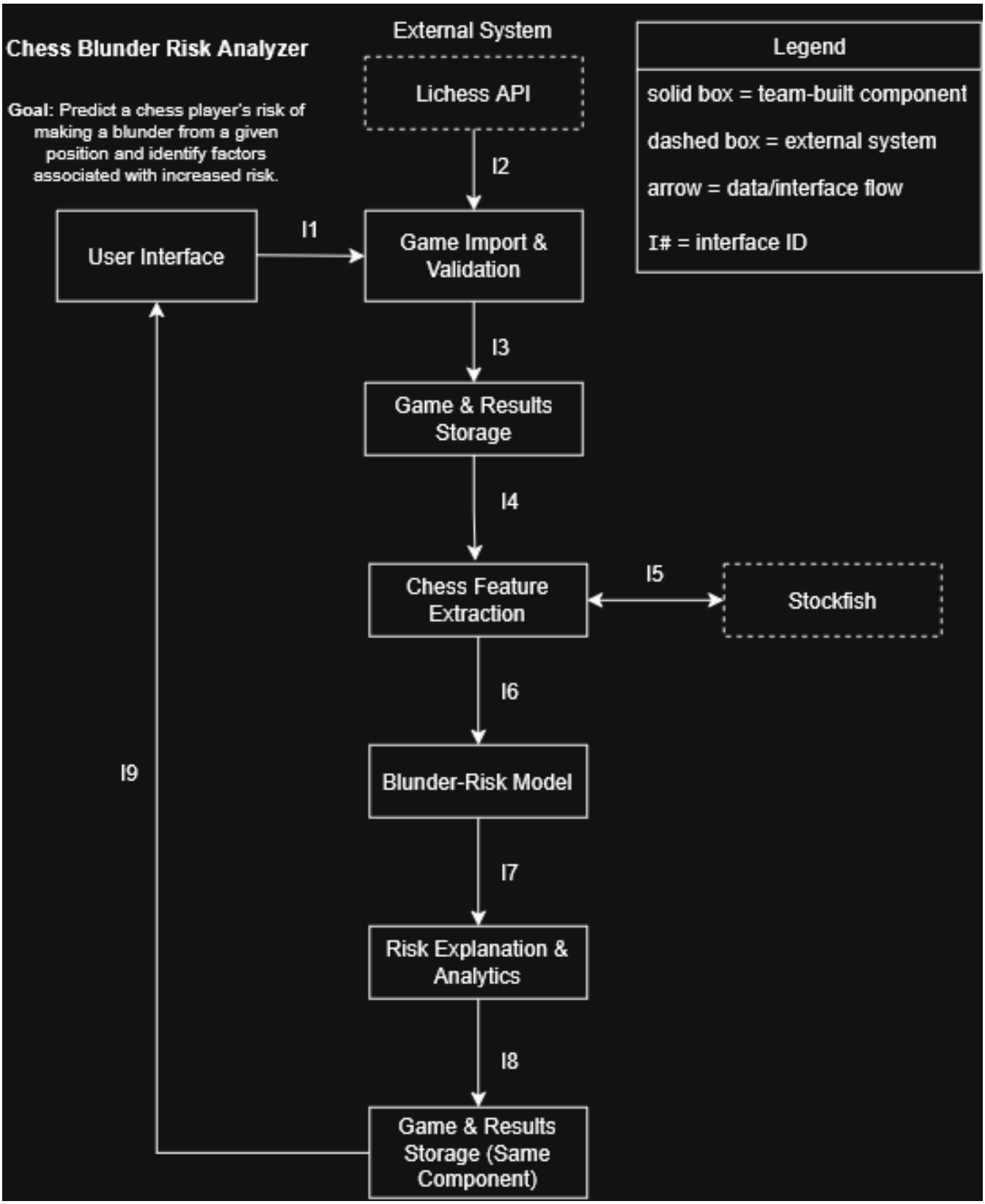
The system is divided into six major internal components:

1. User Interface
2. Game Import and Validation
3. Game and Results Storage
4. Chess Feature Extraction
5. Blunder-Risk Model
6. Risk Explanation and Analytics

The system also interacts with two external systems:

- Lichess API
- Stockfish Chess Engine

A rough structure for the diagram is:


Chess Blunder Risk Analyzer
Goal: Predict a chess player's risk of making a blunder from a given position and identify factors associated with increased risk.
External System
Lichess API
Legend
solid box = team-built component
dashed box = external system
arrow = data/interface flow
I# = interface ID
User Interface
I1
Game Import & Validation
I2
I3
Game & Results Storage
I4
Chess Feature Extraction
I5
Stockfish
I6
Blunder-Risk Model
I7
Risk Explanation & Analytics
I8
Game & Results Storage (Same Component)
I9

3. Component Responsibility Table

Component	Responsibility	Interfaces In	Interfaces Out	Primary Owner
User Interface	Allows users to submit games for analysis and view game-level and move-level results.	I9	I1	Nate Poppe
Game Import and Validation	Obtains completed games from Lichess or uploaded PGN files and validates them before analysis.	I1, I2	I3	Nate Poppe
Game and Results Storage	Stores imported games, progress state, risk scores, explanations, and the model version associated with each analysis.	I3, I8	I4, I9	Nate Poppe
Chess Feature Extraction	Converts chess positions and game metadata into the features required by the blunder-risk model.	I4, I5	I5, I6	Braden Hayes
Blunder-Risk Model	Estimates the probability that a player will make a blunder from an analyzed position.	I6	I7	Braden Hayes
Risk Explanation and Analytics	Converts model outputs into understandable risk factors and aggregates move-level predictions into game or player-level insights.	I7	I8	Braden Hayes

This ownership division places the modeling and statistical analysis primarily under Braden, while Nate primarily owns the user-facing application, game-import pipeline, and persistence layer.

4. Interface Specification Table

Interface Definitions

ID	From → To	Inputs / Outputs	Format	Protocol	Error Handling
I1	User Interface → Game Import and Validation	Lichess username, time control, game count, or uploaded PGN file	JSON / PGN	HTTPS	Invalid input is rejected and an error message is returned to the UI.
I2	Lichess API → Game Import and Validation	Game IDs, PGN data, ratings, clock data, game status, metadata	JSON / PGN	HTTPS REST	Rate-limit errors stop the import while preserving previously saved games. The import can later resume without downloading duplicate games.
I3	Game Import and Validation → Game and Results Storage	Validated game record, game ID, player metadata, move history	Structured JSON	Internal API	Invalid or active games are rejected before storage and analysis.
I4	Game and Results Storage → Chess Feature Extraction	Stored game, positions, ratings, clock values, move metadata	Structured JSON	Internal API	Missing required data causes the position or game to be marked incomplete and excluded from scoring.
I5	Chess Feature Extraction ↔ Stockfish	Input: FEN position and analysis request. Output: evaluation, best move, candidate moves, engine metrics	FEN / UCI text	UCI	Engine timeout or failure causes the affected position to be marked as unavailable or retried.
I6	Chess Feature Extraction →	Numerical feature vector representing	JSON / numeric array	Internal API or	Missing required model features cause the position to be rejected or handled according to the

ID	From → To	Inputs / Outputs	Format	Protocol	Error Handling
	Blunder-Risk Model	the current position and game context		function call	model's preprocessing rules.
I7	Blunder-Risk Model → Risk Explanation and Analytics	Risk score, model version, feature values, optional feature contribution information	JSON	Internal API	Failed inference produces an unavailable score and is logged rather than displaying an incorrect prediction.
I8	Risk Explanation and Analytics → Game and Results Storage	Move number, risk score, risk factors, model version, engine-best move	JSON	Internal API	Failed writes are logged and the result is not marked complete until storage succeeds.
I9	Game and Results Storage → User Interface	Games, top-risk moves, move scores, risk explanations, progress information	JSON	HTTPS	Missing or incomplete results are displayed with an appropriate status message rather than being presented as complete.

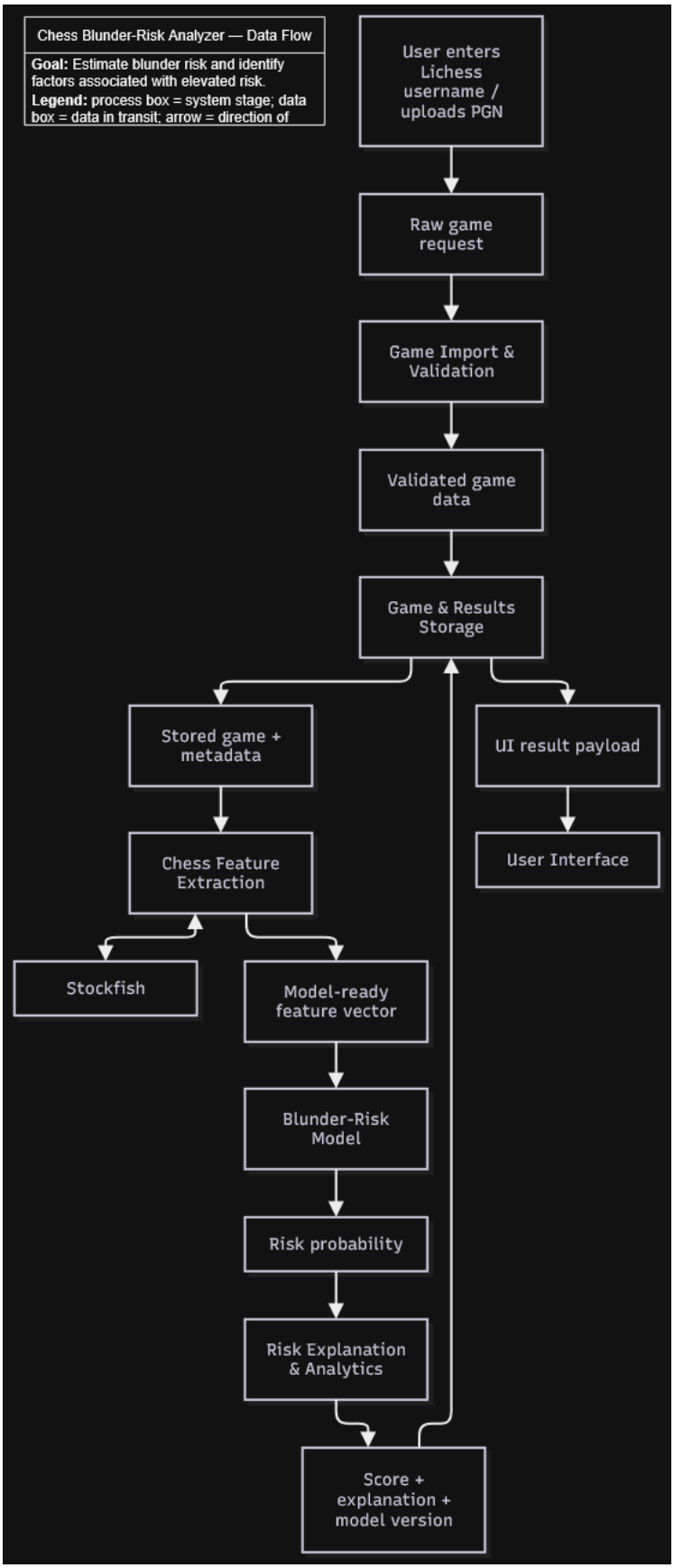
The interface design supports the current use case in which a player can import 1–50 recent games, see progress as they are processed, and later view the three highest-risk moves in each analyzed game.

5. Data-Flow Diagram

The primary data flow begins when a player requests analysis of one or more completed games.

At the import stage, the system verifies that a game has already finished before allowing engine analysis. This behavior supports the current fair-play requirement that the tool must not provide assistance for games that are still being played.

The current acceptance criteria also impose a timing requirement: analysis of 20 recent games should return all games and their three highest-risk moves within **180 seconds** of confirmation.


Chess Blunder-Risk Analyzer — Data Flow
Goal: Estimate blunder risk and identify factors associated with elevated risk.
Legend: process box = system stage; data box = data in transit; arrow = direction of
User enters Lichess username / uploads PGN
Raw game request
Game Import & Validation
Validated game data
Game & Results Storage
Stored game + metadata
UI result payload
Chess Feature Extraction
User Interface
Stockfish
Model-ready feature vector
Blunder-Risk Model
Risk probability
Risk Explanation & Analytics
Score + explanation + model version

This supports the existing requirement that already downloaded games are preserved and are not downloaded again after a rate-limit interruption.

6. Architecture Pattern and Justification

The proposed architecture combines a **client-server architecture** with a **data-processing pipeline**.

The client-server pattern governs communication between the user interface and the backend system. The user interface is responsible for collecting user requests and displaying results, while computationally intensive operations such as game analysis, feature extraction, machine-learning inference, and persistence occur on the backend.

The backend analysis process follows a pipeline architecture:

Game Import → Validation → Feature Extraction → Risk Prediction → Risk Explanation → Results

Fit to the Problem

The pipeline pattern fits the project because each chess game naturally moves through several distinct transformations. Raw game data must first be validated, converted into chess positions, analyzed by an engine, transformed into model-ready features, scored by the risk model, and finally converted into user-facing results.

Team Skills

The architecture divides responsibilities in a way that matches the team's interests and experience. Nate will focus primarily on the user-facing application, game import, and persistence, while Braden will focus primarily on feature engineering, statistical and machine-learning modeling, and prediction analysis.

Performance and Timing

Chess-engine analysis is expected to be one of the most computationally expensive stages of the pipeline. Keeping this processing on the backend avoids performing intensive computation in the user interface and allows the system to control engine-analysis settings. Performance must also be sufficient to satisfy the current requirement of processing 20 games and identifying their highest-risk moves within 180 seconds.

Scalability

The pipeline allows games and positions to be processed independently, which provides a path toward parallel analysis if additional performance is required. Storage also allows previously analyzed games to be reused rather than repeatedly processing identical input.

Hardware Constraints

The system does not require specialized physical hardware. Development and initial deployment can be performed on standard personal computers or general-purpose server infrastructure. Computational cost is primarily associated with Stockfish evaluation and machine-learning inference.

Rejected Alternative: Microservices

A microservices architecture was considered but rejected for the initial system. Although individual components such as Stockfish analysis and model inference could eventually operate as independent services, the added deployment, networking, and service-management complexity is not justified for a two-person capstone team. A modular client-server system with a clear internal processing pipeline provides sufficient separation while remaining manageable within the project timeline.

7. Decision Log

Decision	Alternatives Considered	Selected Option	Reason
Primary purpose of the ML system	Predict the strongest chess move; classify tactical motifs; predict human blunder risk	Predict human blunder risk	The project is intended to model human decision difficulty rather than duplicate the functionality of an existing chess engine.
Primary model output	Binary blunder/not-blunder classification; continuous risk probability	Blunder-risk probability	A probability communicates the degree of risk and enables statistical evaluation through calibration in

Decision	Alternatives Considered	Selected Option	Reason
			addition to classification performance.
Architecture	Microservices; monolithic application; client-server with processing pipeline	Client-server + pipeline	The selected architecture matches the sequential analysis workflow without introducing unnecessary deployment complexity.
Chess evaluation source	Build custom chess evaluation logic; use an existing chess engine	Use Stockfish as an external system	Developing a competitive chess engine is outside the project's goal. Stockfish allows development effort to focus on modeling human mistakes.
Game input method	Manual board entry; PGN upload only; Lichess import only; both import methods	Lichess import and PGN upload	Supporting both allows convenient analysis of online games while still permitting users to analyze games from other sources.
Modeling scope	General-purpose AI chess coach; tactical motif classifier; focused blunder-risk analyzer	Blunder-risk analyzer	The narrower scope provides a measurable ML target while avoiding the difficulty of generating reliable general-purpose chess coaching explanations.
Responsibility split	Divide each subsystem evenly between both team	Braden owns most modeling/analytics work; Nate owns most UI/import/storage work	Dividing ownership by technical area gives each component a clear primary owner and reduces overlapping

Decision	Alternatives Considered	Selected Option	Reason
	members; divide by technical area		implementation responsibility.